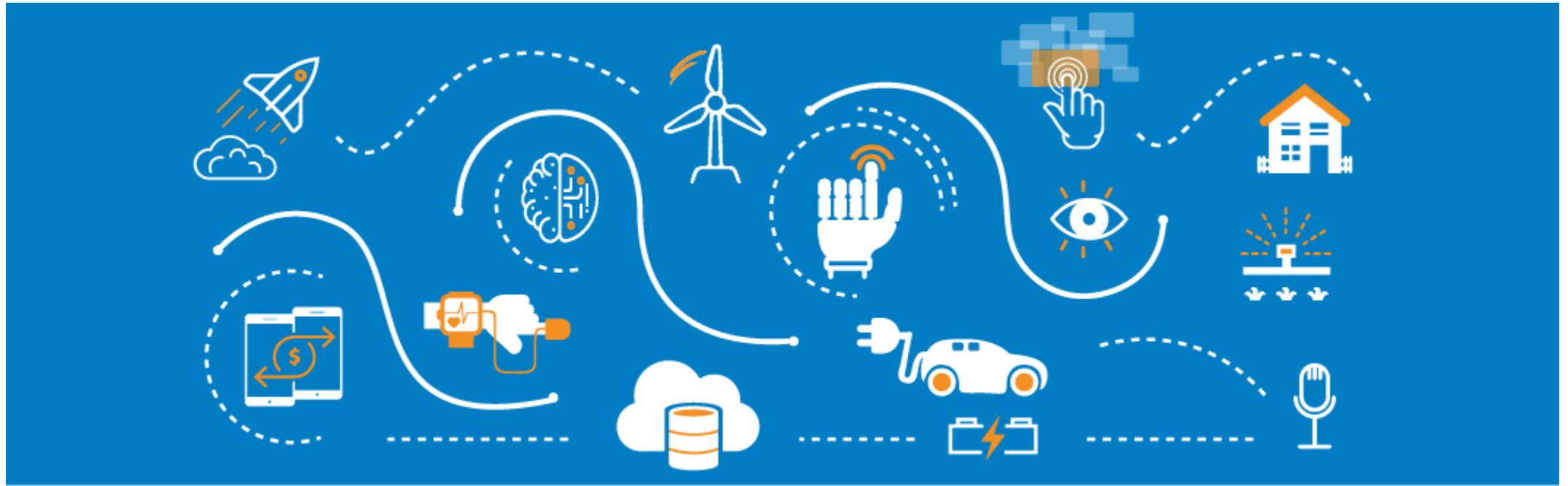




Mathematics For Machine Learning

Kien Tran

Contents

1 **Linear Algebra**

2 **Multivariable Calculus**

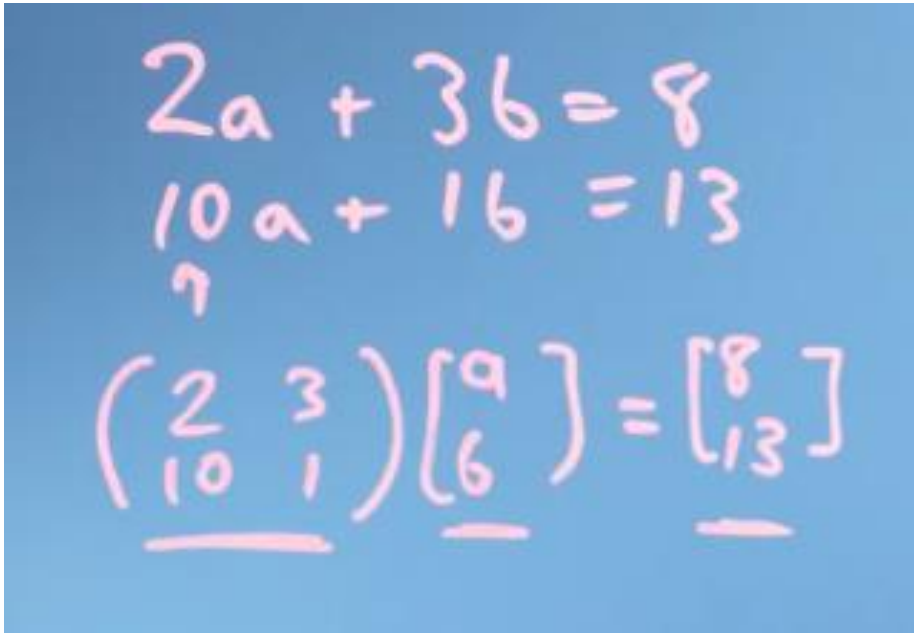
3 **Descriptive Statistics**

4 **Correlation and Regression**

5 **Probability and Probability Distribution**

1. Linear Algebra

1.1. Motivations for Linear Algebra



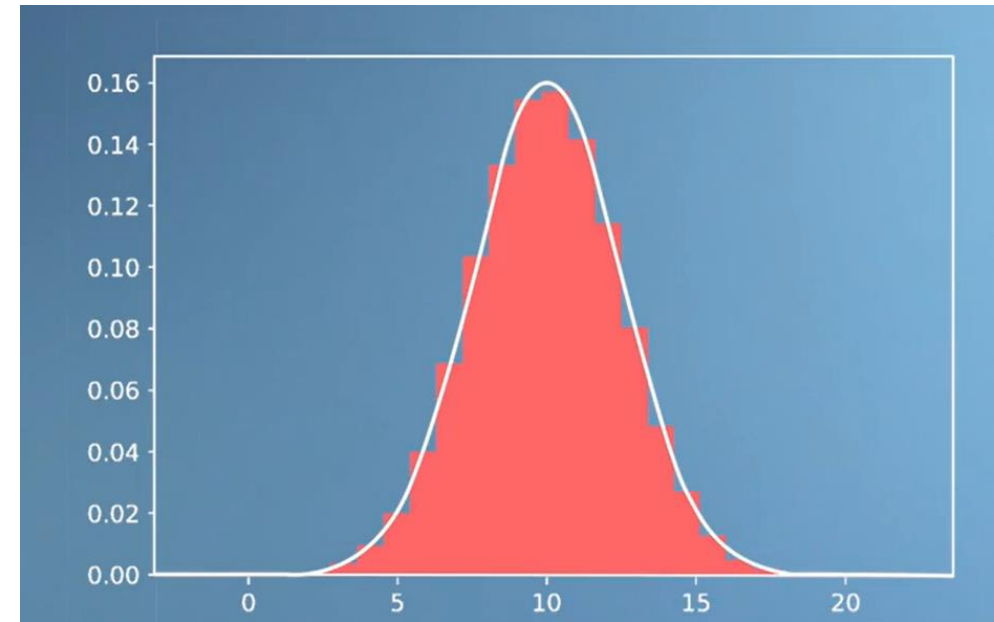
Handwritten equations and matrix representation for buying apples and bananas:

$$\begin{aligned} 2a + 3b &= 8 \\ 10a + 1b &= 13 \end{aligned}$$

Matrix representation:

$$\begin{pmatrix} 2 & 3 \\ 10 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} 8 \\ 13 \end{pmatrix}$$

Pic 1. Compute cost of buying apple and banana in store



Pic 2. Find optimized line in histogram of distributed data

1. Linear Algebra

1.2. Vectors

1.2.1 Definition

Vector is defined under the operation of summation and the multiplication by a scalar

$$\mathbf{x} = (1; 2; 5; 8; 9)$$

```
[1] import numpy as np
```

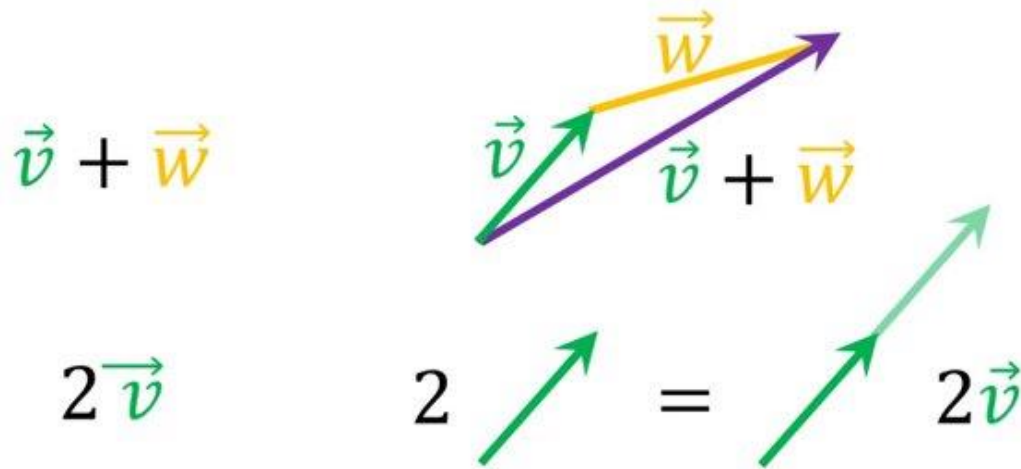
```
[3] x = np.array([1, 2, 5, 8, 9])  
x
```

```
array([1, 2, 5, 8, 9])
```

1. Linear Algebra

1.2. Vectors

1.2.2 Vector Operations



```
[5] import numpy as np
x = np.array([1, 2, 5, 8, 9])
y = np.array([1, 2, 4, 6, 7])
print("x + y: ", x + y)
print("x - y: ", x - y)
print("x * y: ", x * y)
```

```
x + y: [ 2  4  9 14 16]
x - y: [ 0  0  1  2  2]
x * y: [ 1  4 20 48 63]
```

```
x = np.array([1, 2, 5, 8, 9])
print("x + 5: ", x + 5)
print("x - 5: ", x - 5)
print("x * 5: ", x * 5)
```

```
x + 5: [ 6  7 10 13 14]
x - 5: [-4 -3  0  3  4]
x * 5: [ 5 10 25 40 45]
```

1. Linear Algebra

1.3. Matrices

1.3.1 Definition

Matrix is a rectangular array of numbers (or other mathematical objects), called the entries of the matrix

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}.$$

```
import numpy as np
A = np.array([[1, 2, 3],
              [3, 2, 1]])
```

```
# shape của matrix A
A.shape
```

```
(2, 3)
```

1. Linear Algebra

1.3. Matrices

1.3.2 Matrix operations

Element-wise

$$\mathbf{A} \odot \mathbf{B} = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} & \dots & a_{1n}b_{1n} \\ a_{21}b_{21} & a_{22}b_{22} & \dots & a_{2n}b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}b_{m1} & a_{m2}b_{m2} & \dots & a_{mn}b_{mn} \end{bmatrix}$$

```
import numpy as np
A = np.array([[1, 2, 3],
              [3, 2, 1]])

B = np.array([[2, 1, 2],
              [1, 3, 0]])

print("A*B: \n", A*B)
print("A-B: \n", A-B)
print("A+B: \n", A+B)
```

```
A*B:
[[2 2 6]
 [3 6 0]]
A-B:
[[-1  1  1]
 [ 2 -1  1]]
A+B:
[[3 3 5]
 [4 5 1]]
```

1. Linear Algebra

1.3. Matrices

1.3.2 Matrix operations

Matrix multiplication

$$\mathbf{A}_{m \times n} \mathbf{B}_{n \times p} = \begin{bmatrix} \mathbf{A}^{(1)} \mathbf{B}_1 & \mathbf{A}^{(1)} \mathbf{B}_2 & \dots & \mathbf{A}^{(1)} \mathbf{B}_p \\ \mathbf{A}^{(2)} \mathbf{B}_1 & \mathbf{A}^{(2)} \mathbf{B}_2 & \dots & \mathbf{A}^{(2)} \mathbf{B}_p \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{A}^{(m)} \mathbf{B}_1 & \mathbf{A}^{(m)} \mathbf{B}_2 & \dots & \mathbf{A}^{(m)} \mathbf{B}_p \end{bmatrix}_{m \times p}$$

```
import numpy as np
A = np.array([[1, 2, 3],
              [3, 2, 1]])
```

```
B = np.array([[2, 1],
              [1, 3],
              [1, 1]])
```

```
A@B
```

```
array([[ 7, 10],
       [ 9, 10]])
```


1. Linear Algebra

1.3. Matrices

1.3.3 Inverse Matrix

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}_n$$

Ex: $\mathbf{A}\mathbf{x} = \mathbf{b}$

$$\begin{aligned}\mathbf{A}\mathbf{x} &= \mathbf{b} \\ \Leftrightarrow \underbrace{\mathbf{A}^{-1}\mathbf{A}}_{\mathbf{I}}\mathbf{x} &= \mathbf{A}^{-1}\mathbf{b} \\ \mathbf{I}\mathbf{x} &= \mathbf{A}^{-1}\mathbf{b} \\ \mathbf{x} &= \mathbf{A}^{-1}\mathbf{b}\end{aligned}$$

```
import numpy as np
```

```
A = np.array([[1, 2, 3],  
              [3, 2, 1],  
              [4, 2, 2]])
```

```
A_inv = np.linalg.pinv(A)  
print(A_inv)
```

```
[[-0.25 -0.25  0.5 ]  
 [ 0.25  1.25 -1.  ]  
 [ 0.25 -0.75  0.5 ]]
```

1. Linear Algebra

1.3. Matrices

1.3.4 Matrix in Python

```
import numpy as np
A = np.array([[1, 2, 3],
              [3, 2, 1],
              [4, 2, 2]])
```

```
# Truy cập ma trận có các dòng chẵn
A[::2, :]
```

```
array([[1, 2, 3],
       [4, 2, 2]])
```

```
# Truy cập một index cụ thể, ví dụ dòng 0, 2 của ma trận
A[[0, 2], :]
```

```
array([[1, 2, 3],
       [4, 2, 2]])
```

```
# Truy cập ma trận con từ 2 dòng đầu tiên và 2 cột đầu tiên.
A[:2, :2]
```

```
array([[1, 2],
       [3, 2]])
```

```
] # Truy cập ma trận con từ 2 dòng cuối cùng và 2 cột đầu tiên
A[-2:, :2]
```

```
array([[3, 2],
       [4, 2]])
```

```
] # Truy cập véc tơ con từ dòng thứ 2 và 2 cột cuối cùng.
print(A[2, -2:])
```

```
# Hoặc
A[2:3, -2:][0]
```

```
[2 2]
array([2, 2])
```

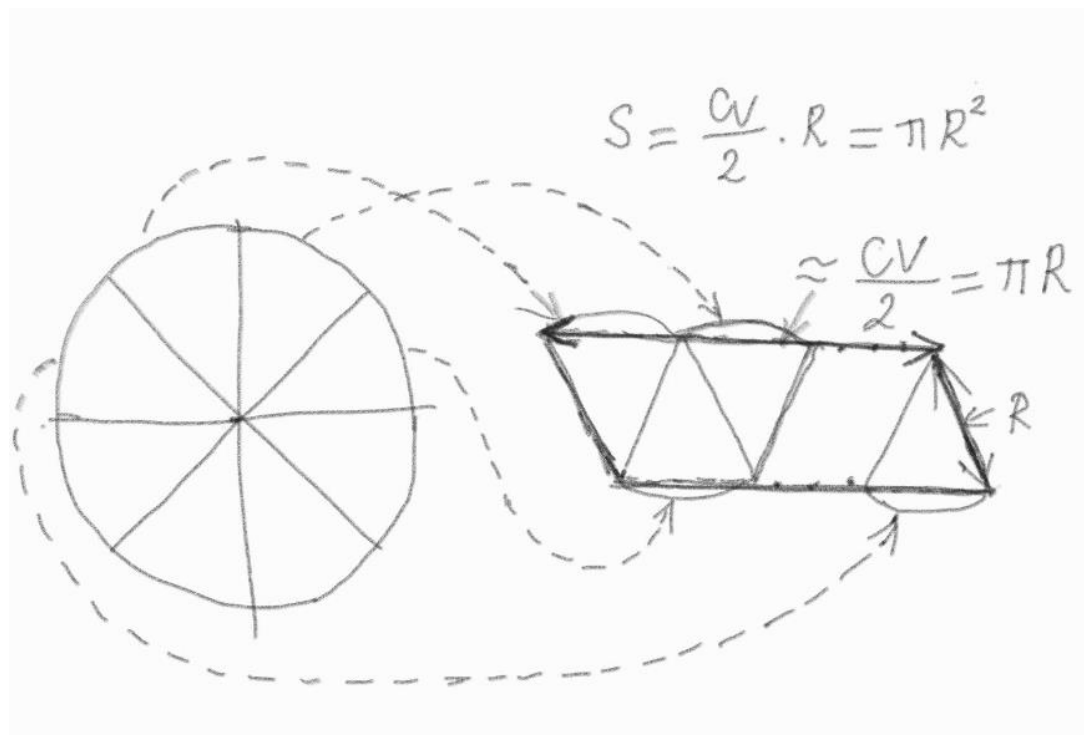
1. Linear Algebra

1.4. Exercises

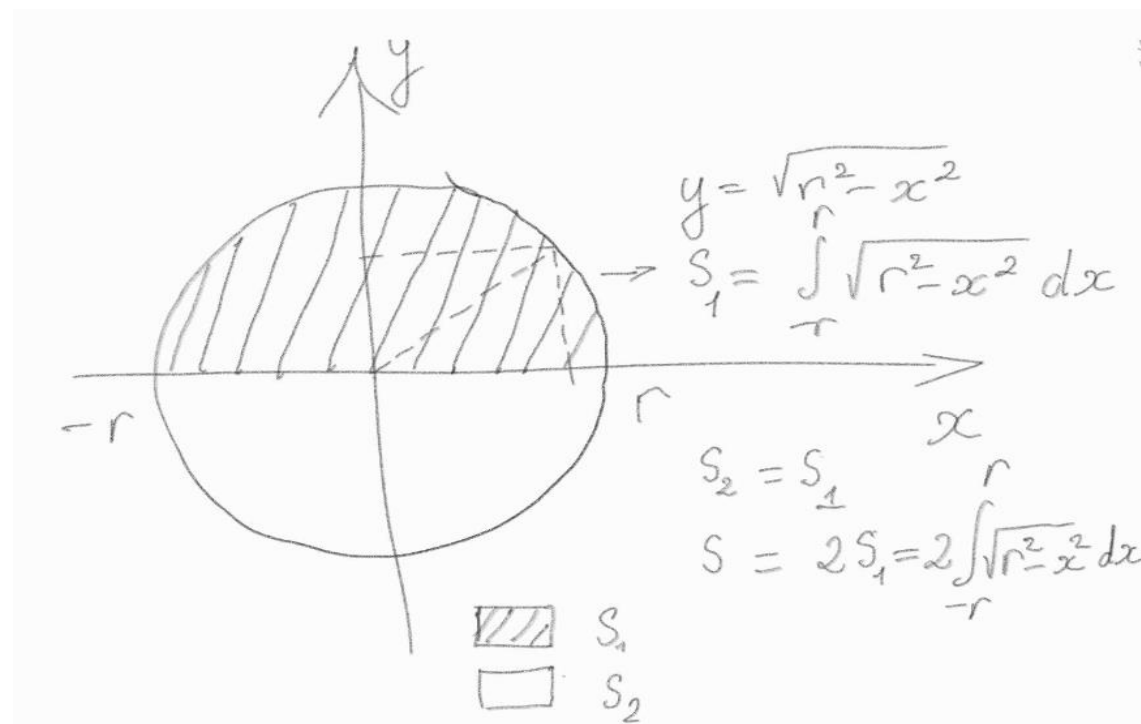
https://drive.google.com/file/d/1eSYMtHj-HastfapJsGRaaKh1Ff09hWJv/view?usp=drive_link

2. Multivariable Calculus

2.1. Integral Calculus



traditional



Infinitesimal Calculus

2. Multivariable Calculus

2.2. Differential Calculus

$$f'(x_0) = \lim_{x \rightarrow x_0} \frac{f(x) - f(x_0)}{x - x_0}$$

```
import torch

# Tính đạo hàm của vận tốc f(x) = x^2+2*x

def _derivative(x):
    f1 = x**2 + 2*x
    delta = 0.001 # giá trị delta ở mẫu
    x0 = x-delta
    f0 = x0**2 + 2*x0
    der = (f1-f0)/delta
    return der

_derivative(10)
```

21.998999999979674

2. Multivariable Calculus

2.3. Gradient descent

Partial derivative

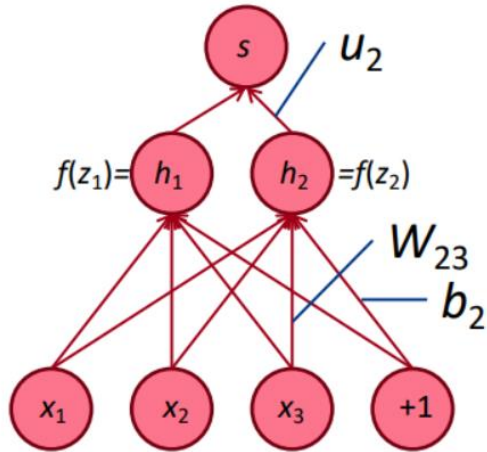
$$\frac{\partial f(x_1, x_2, \dots, x_i, \dots, x_n)}{\partial x_i} = \lim_{\Delta \rightarrow 0} \frac{f(x_1, x_2, \dots, x_i + \Delta, \dots, x_n) - f(x_1, x_2, \dots, x_i, \dots, x_n)}{\Delta}$$

Gradient descent

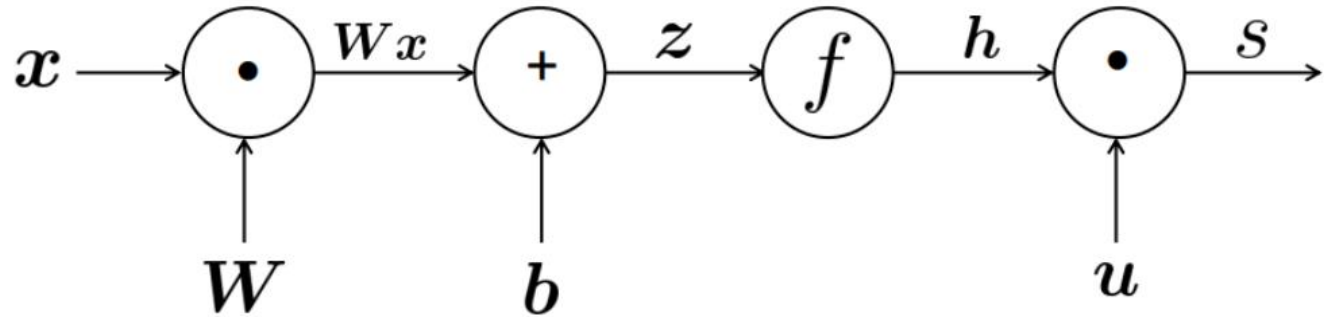
$$\nabla_{\mathbf{w}} f(\mathbf{w}) = \left[\frac{\partial f(\mathbf{w})}{\partial w_1}, \frac{\partial f(\mathbf{w})}{\partial w_2}, \dots, \frac{\partial f(\mathbf{w})}{\partial w_n} \right]^\top$$

2. Multivariable Calculus

2.4. Feed forward



Neural Network



Input: x

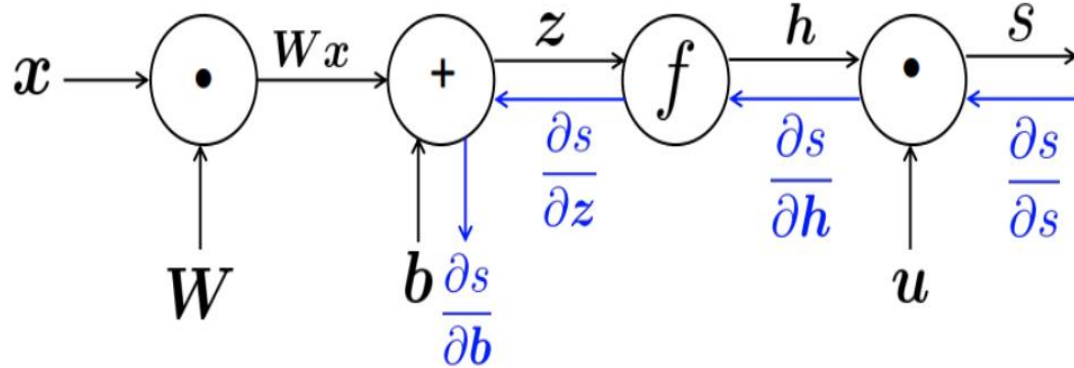
Layer 1: $z = \mathbf{W}x + b$

Activation: $\mathbf{h} = f(z)$

Output: $s = \mathbf{u}^T \mathbf{h}$

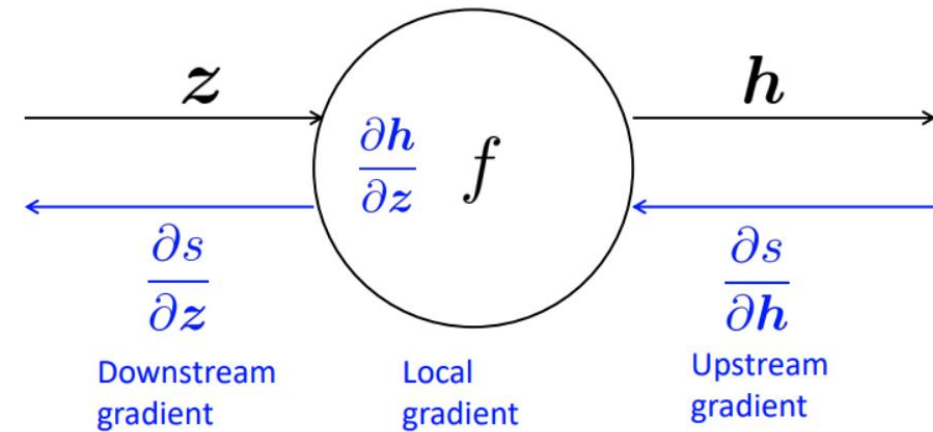
2. Multivariable Calculus

2.5. Backpropagation



backpropagation

Chain rule
$$\frac{\delta s}{\delta z} = \frac{\delta s}{\delta \mathbf{h}} \frac{\delta \mathbf{h}}{\delta z}$$

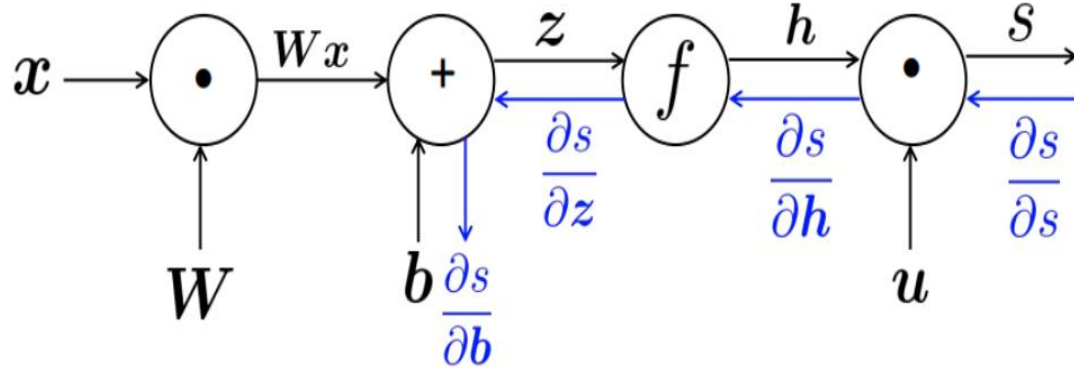


$$\mathbf{h} = f(\mathbf{z})$$

$$\text{downstream} = \text{upstream} \times \text{local}$$

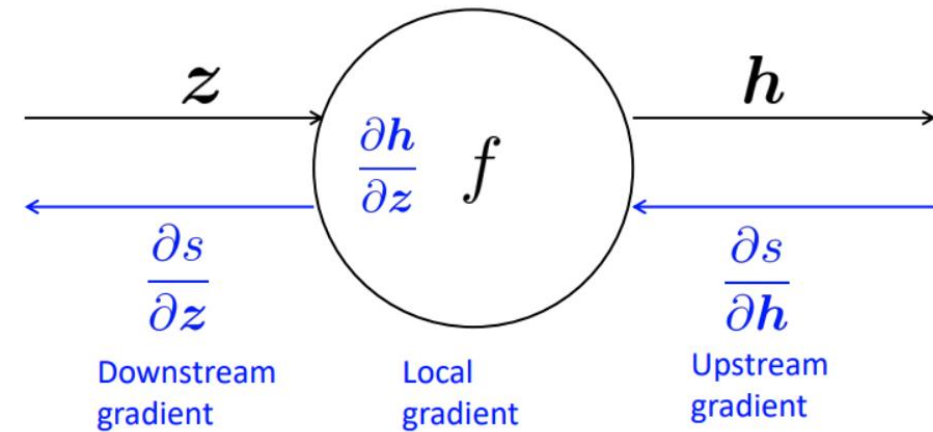
2. Multivariable Calculus

2.6. Backpropagation



backpropagation

Chain rule
$$\frac{\delta s}{\delta z} = \frac{\delta s}{\delta \mathbf{h}} \frac{\delta \mathbf{h}}{\delta z}$$



$$\mathbf{h} = \mathbf{f}(\mathbf{z})$$

$$\text{downstream} = \text{upstream} \times \text{local}$$

2. Multivariable Calculus

2.7. Exercise

Run notebooks below to understand gradient

[The Sandpit](#)

[The Sandpit 2](#)

[Gradient Descent The Sandpit](#)

3. Descriptive Statistics

3.1. Classification of statistical data

- Variables are characteristics of someone or something
- Case (or observation) is someone or something

For example: The case is the students.

The variables are the student's hair color, height, and grades

Two types of variables:

- **Qualitative variables** (categorical variables):
nominal data and ordinal data
- **Quantitative variables:**
Discrete data and Continuous data

LEVELS OF MEASUREMENT					
		Difference	Order	Similar intervals	Meaningful zero point
Categorical	nominal	+	-	-	-
	ordinal	+	+	-	-
Quantitative	interval	+	+	+	-
	ratio	+	+	+	+

DISCRETE set of separate numbers

CONTINUOUS infinite region of values



3. Descriptive Statistics

3.2. Measures for central tendency and dispersion tendency

3.2.1 Mode, Mean and Medium

- Mean:
$$\mu = \frac{\sum x_i}{N}$$
 where N is the number of data in the population

- Medium
$$\bar{x} = \frac{\sum x_i}{n}$$
 where n is the sample size

- Mode

```
import numpy as np
speed = [99,86,87,88,111,86,103,87,94,78,77,85,86]
# Mean
x = np.mean(speed)

print(x)
```

89.76923076923077

```
# Median
x = np.median(speed)
print(x)
```

87.0

```
from scipy import stats
# Mode
x = stats.mode(speed)
print(x)
```

ModeResult(mode=86, count=3)

3. Descriptive Statistics

3.2. Measures for central tendency and dispersion tendency

3.2.2 Variance and Standard Deviation

- Variance: $\sigma^2 = \frac{\sum (x - \mu)^2}{N}$ where N is the number of data in the population

- Standard deviation $\sigma = \sqrt{\frac{\sum (x - \mu)^2}{N}}$ where n is the sample size

```
import numpy as np

# init random x with length=100
x = np.random.randn(100)
# variance của x
np.var(x, ddof=1)
```

1.0512332693910436

```
# standard deviation của x
np.std(x, ddof=1)

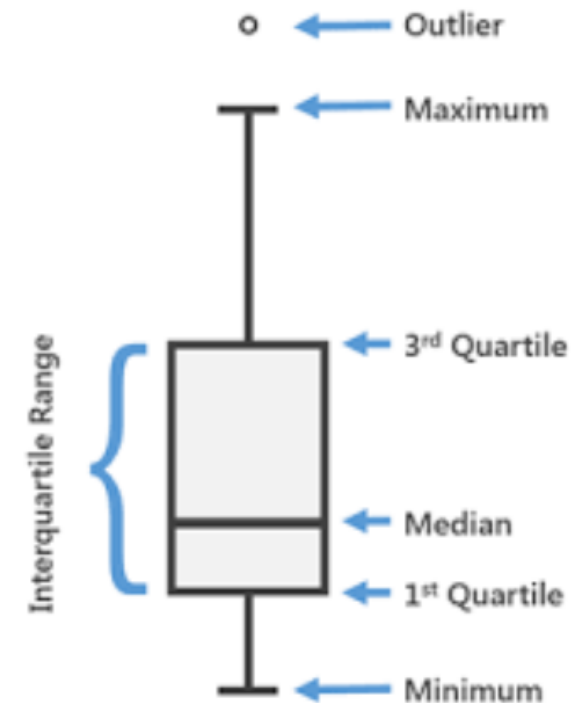
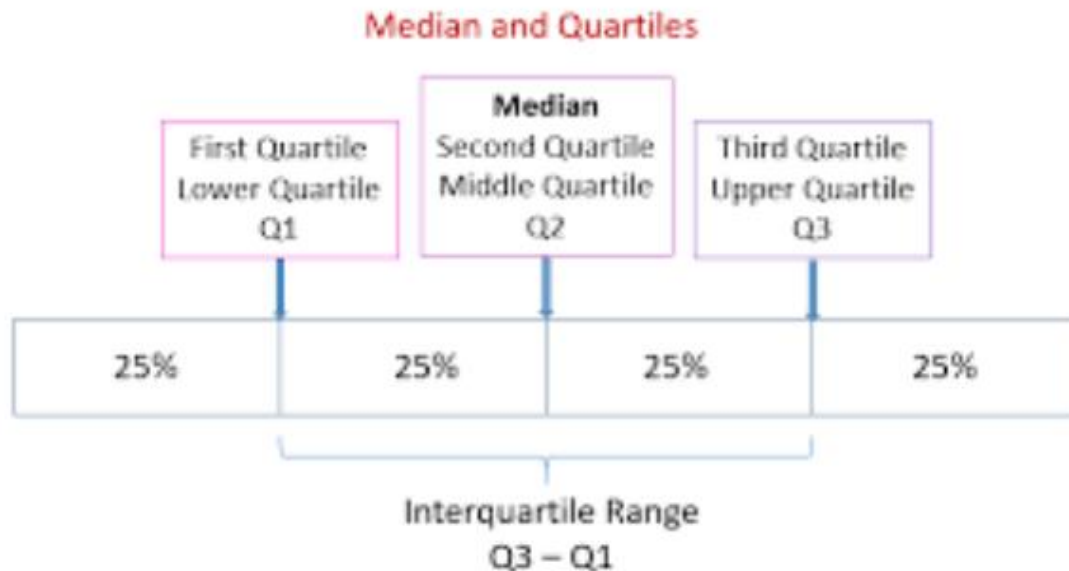
# or
x.std(ddof=1)
```

1.0252966738417928

3. Descriptive Statistics

3.2. Measures for central tendency and dispersion tendency

3.2.3 Amplitude, Spread and Box-plot



$$IQR = Q3 - Q1$$

$$\text{Left_outlier} < Q1 - 1.5IQR$$

$$\text{Right_outlier} > Q3 + 1.5IQR$$

3. Descriptive Statistics

3.3. Exercise

[1. Mean, median and mode](#)

[2. Weighted Mean](#)

[3. Quartiles](#)

[4. Interquartile range](#)

[5. Standard Deviation](#)

[6. Standard Deviation Puzzle 1](#)

4. Correlation and Regression

4.1. Correlation coefficient r

Formula to calculate correlation coefficient r: $r = \frac{\sum z_x z_y}{n - 1}$, where $z_x = \frac{x - \bar{x}}{s_x}$; $z_y = \frac{y - \bar{y}}{s_y}$ and n is the number of data pairs (x,y) in the sample

z (z-score): the number of standard deviations a given data point is from the average

- $r > 0$ corresponds to a positive correlation
- $r < 0$ corresponds to negative correlation
- r only accepts values in the range $[-1;1]$; The closer the absolute value of r is to 1, the stronger the linear correlation is, and when $|r|=1$, the data sample has the strongest linear correlation (all data lie on a straight line).
- $r=0$ corresponds to the case of uncorrelated data samples.
- If you swap the roles of x and y, the value of r does not change.

4. Correlation and Regression

4.2. Regression problem

The slope coefficient (also known as the slope coefficient of the regression line) b and intersect it with the axis O by a to build the linear regression line equation $y = a + bx$:

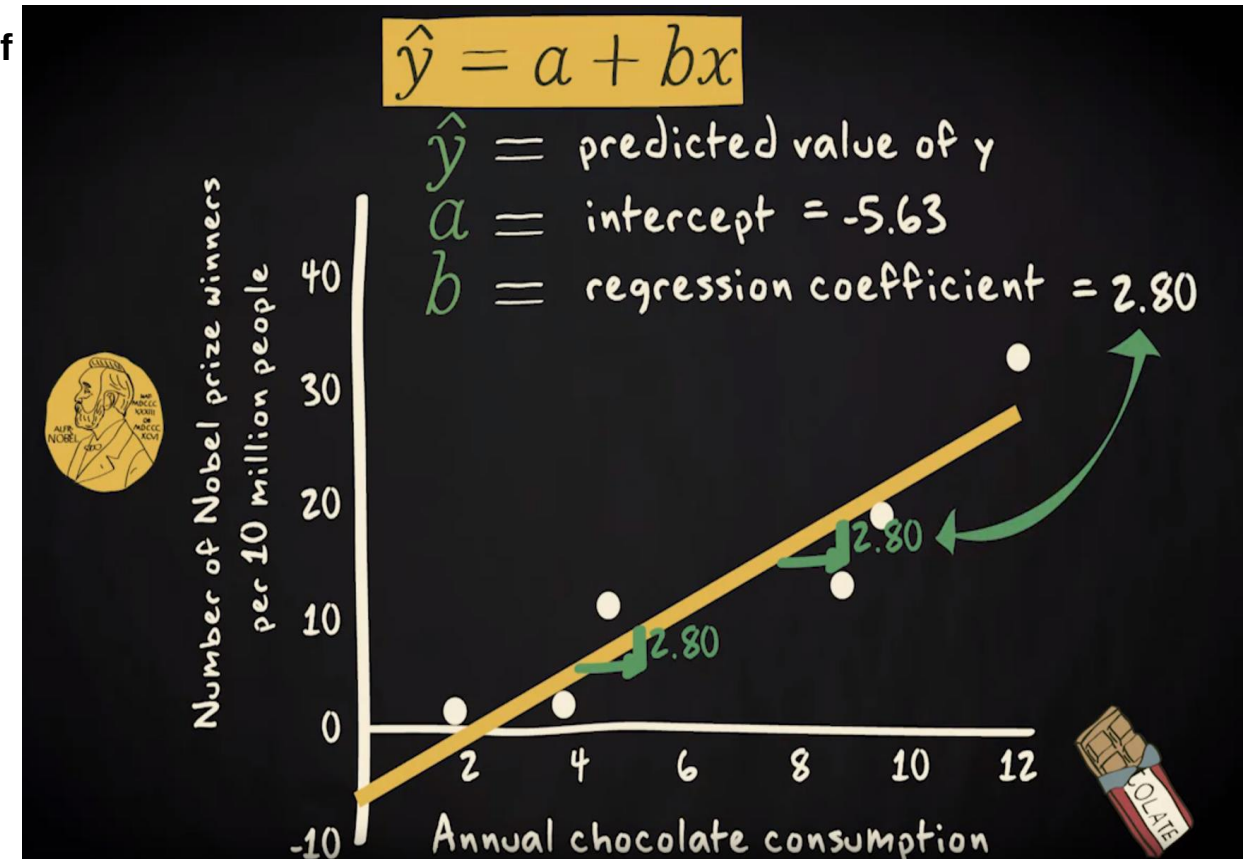
- Slope coefficient of the regression line:

$$b = r * \frac{s_y}{s_x}$$

- Intersection of the regression line with the vertical axis:

$$a = \bar{y} - b\bar{x}$$

- $r > 0$ corresponds to the covariate regression line
- $r < 0$ corresponds to the inverse regression line



5. Probability and Probability Distribution

5.1. Probability

Randomness

$$0 \leq \text{Probability} \leq 1$$

$$\sum \text{Probabilities} = 1$$

The sample space Ω is the set of all possibilities that can occur when conducting an experiment.

Event A is the phenomenon we are interested in, A is a subset of the sample space.

Probability of event A occurring: $P(A) = \frac{|A|}{|\Omega|}$

• **Joint probability:** probability of an event formed when both characteristics occur together (intersection of two characteristics)

• **Marginal probability:** the probability of the outcome of each individual characteristic

• **Conditional probability**

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

5. Probability and Probability Distribution

5.2. Probability Distribution

5.2.1 Gaussian distribution

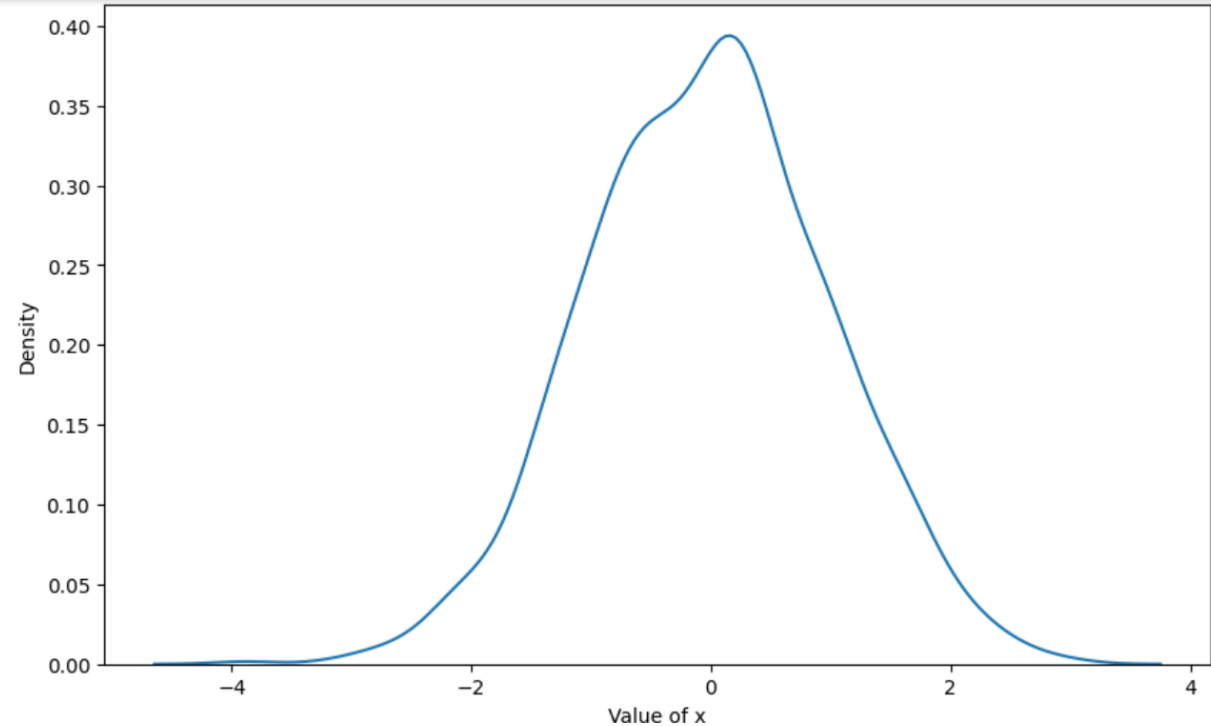
Is a bell-shaped density function and symmetrical about the average value

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

```
import torch
import matplotlib.pyplot as plt
import seaborn as sns

def _plot(x, title, figsize=(10, 6)):
    plt.figure(figsize=figsize)
    sns.kdeplot(x)
    plt.xlabel("Value of x")
    plt.title(title)

x = torch.randn(1000)
_plot(x, "Standard Normal Distribution")
```



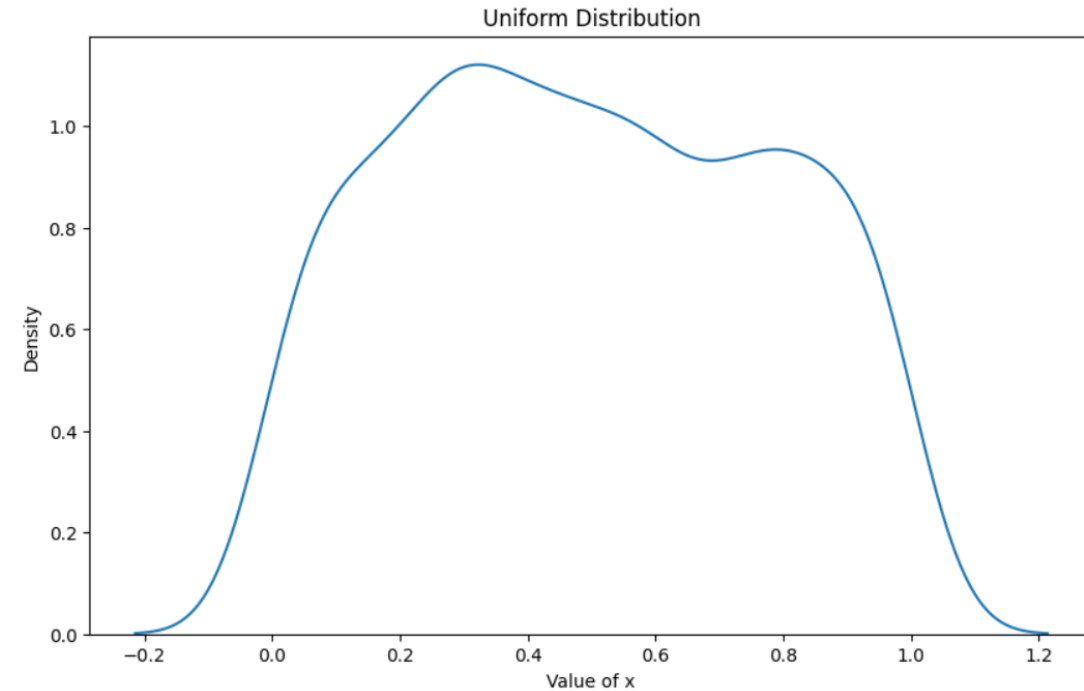
5. Probability and Probability Distribution

5.2. Probability Distribution

5.2.2 Uniform distribution

uniform distribution is a continuous distribution over a segment $[a,b]$

$$f(x) = \begin{cases} \frac{1}{b-a} & \text{if } x \in [a, b] \\ 0 & \text{ifelse} \end{cases}$$



```
x = torch.rand(1000)
_plot(x, "Uniform Distribution")
```

5. Probability and Probability Distribution

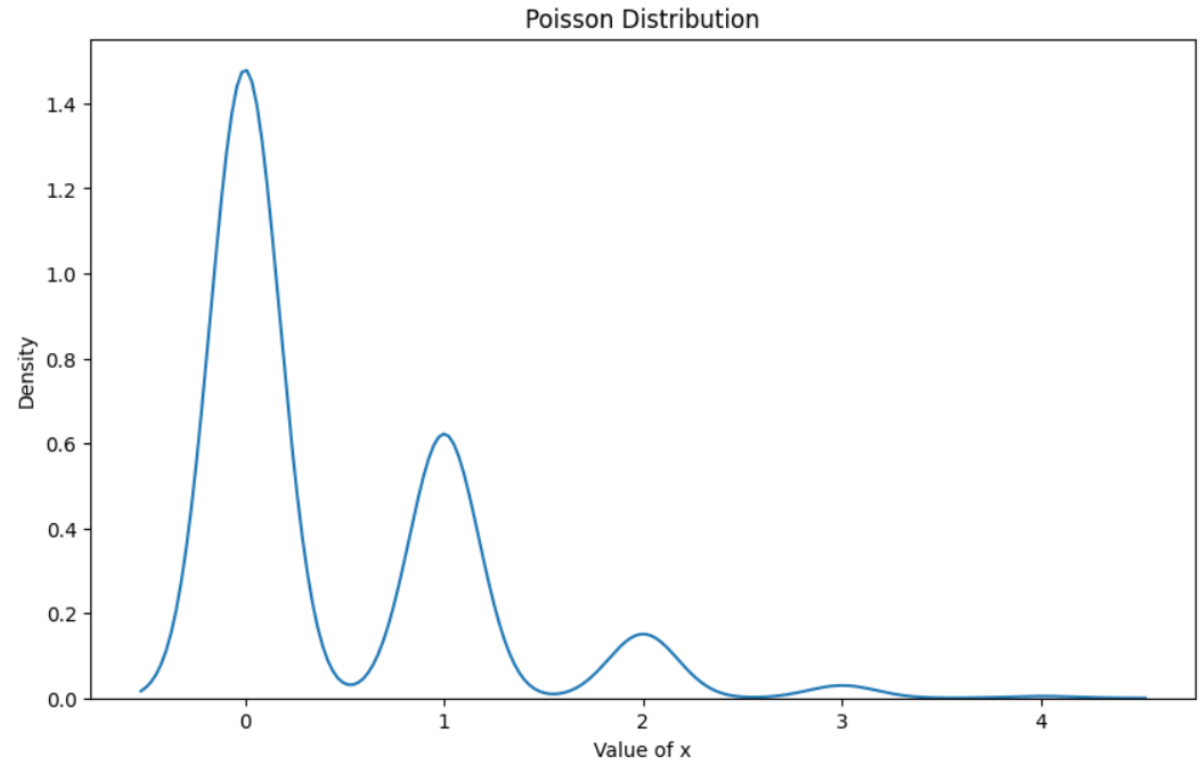
5.2. Probability Distribution

5.2.3 Poisson distribution

Suppose you are sitting in a hospital. Probability to the front you have x people? On average, how long do you have to wait when you get to the hospital? The Poisson distribution will be highly applicable in this case

$$f(k; \lambda) = P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

k : number of events
 λ : expect of distribution



```
rates = torch.rand(1000)
x = torch.poisson(rates)
_plot(x, "Poisson Distribution")
```

5. Probability and Probability Distribution

5.3. Exercise

[1. Probability puzzles 1](#)

[2. Probability puzzles 2](#)

[3. Probability puzzles 3](#)

[4. Probability puzzles 4](#)

[5. Probability puzzles 5](#)

[6. Probability puzzles 6](#)

[7. Normal Distribution](#)

[8. Normal Distribution 2](#)

THANK YOU!



Email: contact@tmainnovation.vn



Website: www.tmainnovation.vn

